

개발된 LLM 연동 웹 애플리케이션

제출일자 : 2025.03.24. (화)

작성 팀원 : 우재현

1. 개요

1.1 목표

Kalpie는 웹 소스 기반 RAG(Retrieval-Augmented Generation) 노트북 서비스이다. 사용자가 URL을 등록하면 자동으로 크롤링·청킹·임베딩하여 벡터 DB에 적재하고, 이를 기반으로 LLM이 출처가 명시된 답변을 생성한다. 추가로 에이전트 기반 리포트 자동 작성 기능을 제공한다.

1.2 주요 기능

기능	설명
노트북 관리	노트북 생성·수정·삭제·고정, 소스를 노트북 단위로 그룹핑
소스 등록	URL 입력 시 Data Service가 크롤링→정제→청킹→임베딩을 자동 수행, 진행 상황 SSE 스트리밍
북마크 동기화	Chrome 확장 프로그램(Bookalpie)에서 북마크를 폴더 구조 그대로 노트북에 전송
RAG 채팅	활성화된 소스를 대상으로 벡터검색→리랭킹→ LLM 답변 생성, 출처 인용 포함
리포트 생성	4단계 에이전트 워크플로우(요구사항 분석→목차→초안→최종)로 구조화 문서 자동 작성
인증	Google OAuth 2.0 및 이메일/비밀번호 JWT 인증

1.3 기술 스택

영역	기술
----	----

Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS 4, Radix UI, Zustand
Backend	FastAPI, Python 3.11+, LangGraph, LangChain, SQLAlchemy 2.0
Data Service	FastAPI, Trafilatura, Playwright, LangChain Text Splitters
Extension	Chrome Manifest V3, React 19, Vite 5, CRXJS, Zustand
Database	PostgreSQL + pgvector (HNSW 인덱스), Alembic 마이그레이션
LLM/ML	GPT-4o-mini(추론), EXAONE-4.0-32B(대체), bge-m3(임베딩), bge-reranker-v2-m3(리랭커)
인프라	RunPod Serverless(GPU 추론), AWS RDS(PostgreSQL)
패키지 관리	uv(Python), pnpm(Frontend/Extension)

2. 설치 및 설정

2.1 사전 요구 사항

항목	버전	설치 확인
Python	3.11 이상	python --version
Node.js	20 이상	node --version
uv	최신	uv --version
pnpm	10 이상	pnpm --version
PostgreSQL	15 이상 (pgvector 확장 포함)	psql --version

2.2 Backend (FastAPI) 실행

```
cd backend          #디렉토리 이동
uv sync             #의존성 설치
.env 설정           #환경변수 설정(2.5절 참고)
uvicorn main:app --reload  # 서버 실행
```

서버 시작 시 SQLAlchemy의 `Base.metadata.create_all()`로 DB 테이블이 자동 생성된다.

주요 의존성:

패키지	버전	용도
fastapi	≥0.128.0	REST API 프레임워크
uvicorn[standard]	≥0.40.0	ASGI 서버
sqlalchemy	≥2.0.46	ORM
asyncpg	≥0.31.0	비동기 PostgreSQL 드라이버
langgraph	≥1.0.8	LLM 에이전트 그래프
langchain-openai	≥1.1.9	OpenAI LLM 연동
langchain-postgres	≥0.0.17	pgvector 벡터 스토어
python-jose[cryptography]	≥3.5.0	JWT 인증
google-auth	≥2.48.0	Google OAuth
alembic	≥1.18.4	DB 마이그레이션

2.3 Frontend (Next.js) 실행

```
cd frontend          # 디렉토리 이동
pnpm install          # 의존성 설치
.env 설정             #환경변수 설정(2.5절 참고)
pnpm dev              # 개발 서버 실행
```

주요 의존성:

패키지	버전	용도
next	16.1.6	React 풀스택 프레임워크
react / react-dom	19.2.3	UI 라이브러리
zustand	5.0.11	클라이언트 상태 관리
@radix-ui/*	최신	접근성 기반 UI 컴포넌트
react-markdown	10.1.0	마크다운 렌더링

react-hook-form + zod	최신	폼 검증
axios	1.13.4	HTTP 클라이언트
@microsoft/fetch-event-source	2.0.1	SSE 스트리밍 수신
tailwindcss	4	유틸리티 CSS

2.4 Data Service 실행

```

cd data                                # 디렉토리 이동
uv sync                                # 의존성 설치
uv run playwright install chromium      # Playwright 브라우저 설치 (최초 1회)
.env 설정                               # 환경변수 설정(2.5절 참고)
uvicorn main:app --reload               # 서버 실행

```

주요 의존성:

패키지	버전	용도
fastapi	≥0.131.0	REST API 프레임워크
trafilatura	≥2.0.0	웹 콘텐츠 추출
playwright	≥1.58.0	동적 페이지 크롤링
langchain-postgres	≥0.0.14	pgvector 벡터 스토어
langchain-text-splitters	≥1.1.1	텍스트 청킹
pgvector	≥0.2.5	벡터 유사도 검색
beautifulsoup4	≥4.14.3	HTML 파싱

2.5 환경변수 설정

Backend .env

변수명	설명
DATABASE_URL	PostgreSQL 동기 연결 문자열
ASYNC_DATABASE_URL	PostgreSQL 비동기 연결 문자열

SECRET_KEY	JWT 서명 키
OPENAI_API_KEY	OpenAI API 키
GOOGLE_CLIENT_ID	Google OAuth Client ID
GOOGLE_CLIENT_SECRET	Google OAuth Client Secret
CHUNKING_CRAWL_URL	Data Service URL
EMBEDDING_MODEL_URL	임베딩 모델 엔드포인트 (RunPod)
RERANKER_MODEL_URL	리랭커 모델 엔드포인트 (RunPod)
CUSTOM_LLM_MODEL_URL	커스텀 LLM 엔드포인트 (RunPod)
RUNPOD_API_KEY	RunPod API 인증 키
BACKEND_CORS_ORIGINS	CORS 허용 출처
REFRESH_TOKEN_EXPIRE_DAYS	리프레시 토큰 만료 일수

Frontend .env

변수명	설명	예시
NEXT_PUBLIC_API_URL	Backend API 기본 URL	http://localhost:3000
NEXT_PUBLIC_GOOGLE_CLIENT_ID	Google OAuth Client ID	xxx.apps.googleusercontent.com

Data Service .env

변수명	설명	예시
DATABASE_URL	PostgreSQL 연결 문자열	postgresql+psycopg://user:pass@host/db

BACKEND_CALLBACK_URL	Backend 콜백 URL	http://localhost:8000/
EMBED_BASE_URL	임베딩 서비스 URL (RunPod)	https://api.runpod.ai/v2/...
EMBED_MODEL	임베딩 모델명	BAAI/bge-m3
RUNPOD_API_KEY	RunPod API 인증 키	rpa_...

2.6 데이터베이스 설정

PostgreSQL에 pgvector를 확장해야 한다.

- CREATE EXTENSION IF NOT EXISTS vector;

Backend와 Data Service 모두 Alembic으로 마이그레이션을 관리한다.

Backend 마이그레이션 적용

- cd backend
- alembic upgrade head

Data Service 마이그레이션 적용

- cd data
- alembic upgrade head

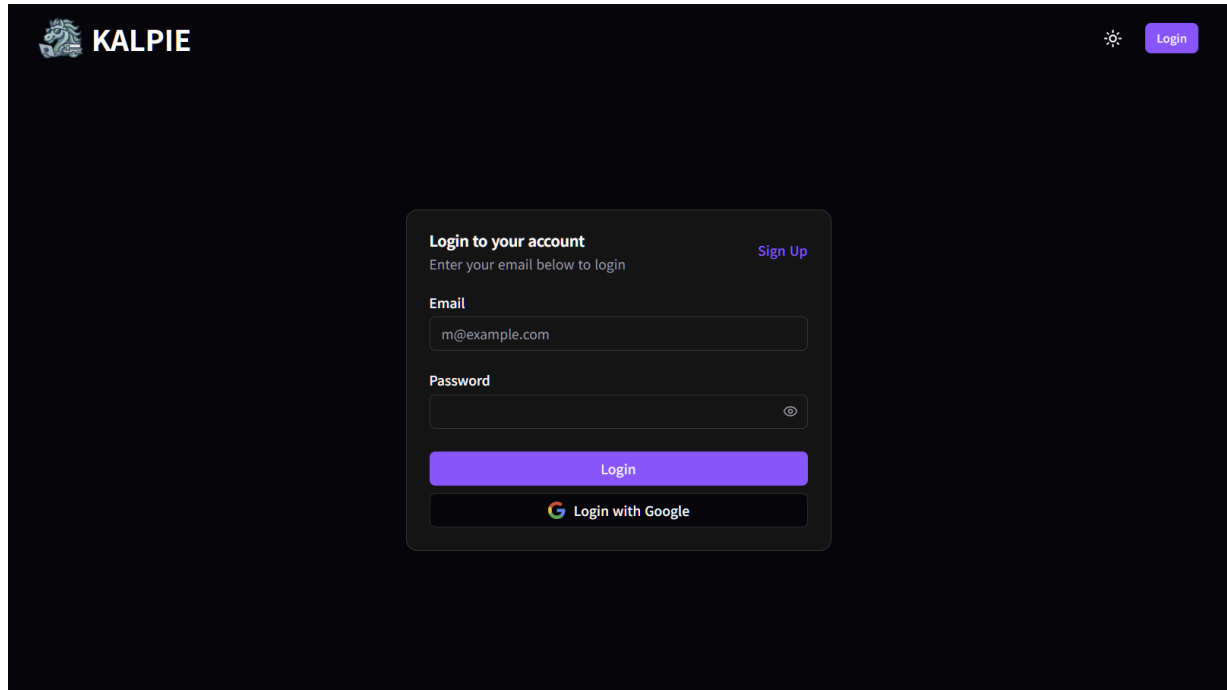
주요 테이블 구조

테이블	설명
users	사용자 계정 (이메일, 비밀번호 해시, Google ID)
notebooks	노트북 (제목, 고정 여부, 소유자)
directories	폴더 구조 (계층적 트리)
sources	소스 URL (제목, URL, 활성 상태, 크롤링 상태)
chats	채팅 이력 (질문, 답변, 소스 메타데이터)
langchain_pg_collection	pgvector 컬렉션 메타데이터
langchain_pg_embedding	벡터 임베딩 데이터 (1024차원, HNSW 인덱스)

3. 기본 사용법

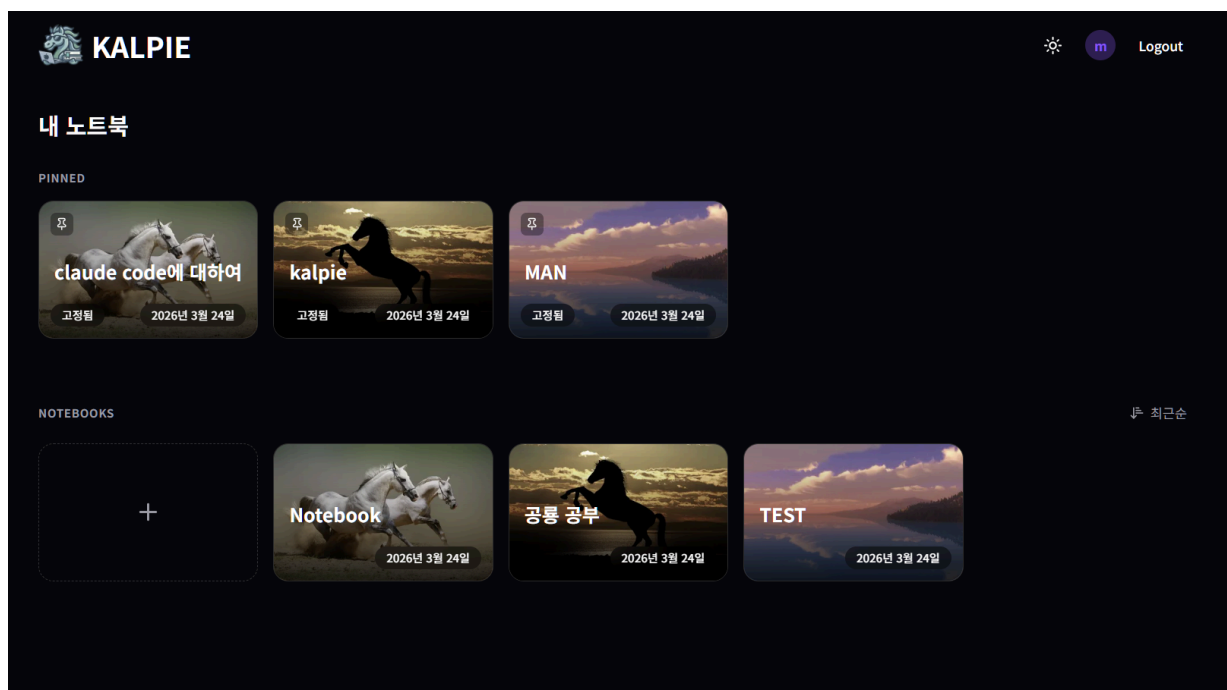
3.1 로그인 / 회원가입

Google OAuth 로그인 또는 이메일/비밀번호로 계정을 생성한다. 로그인 시 JWT 액세스 토큰이 발급되며, 리프레시 토큰은 HttpOnly 쿠키에 저장된다.



3.2 메인 화면 (노트북 목록)

로그인 후 "내 노트북" 대시보드가 표시된다.

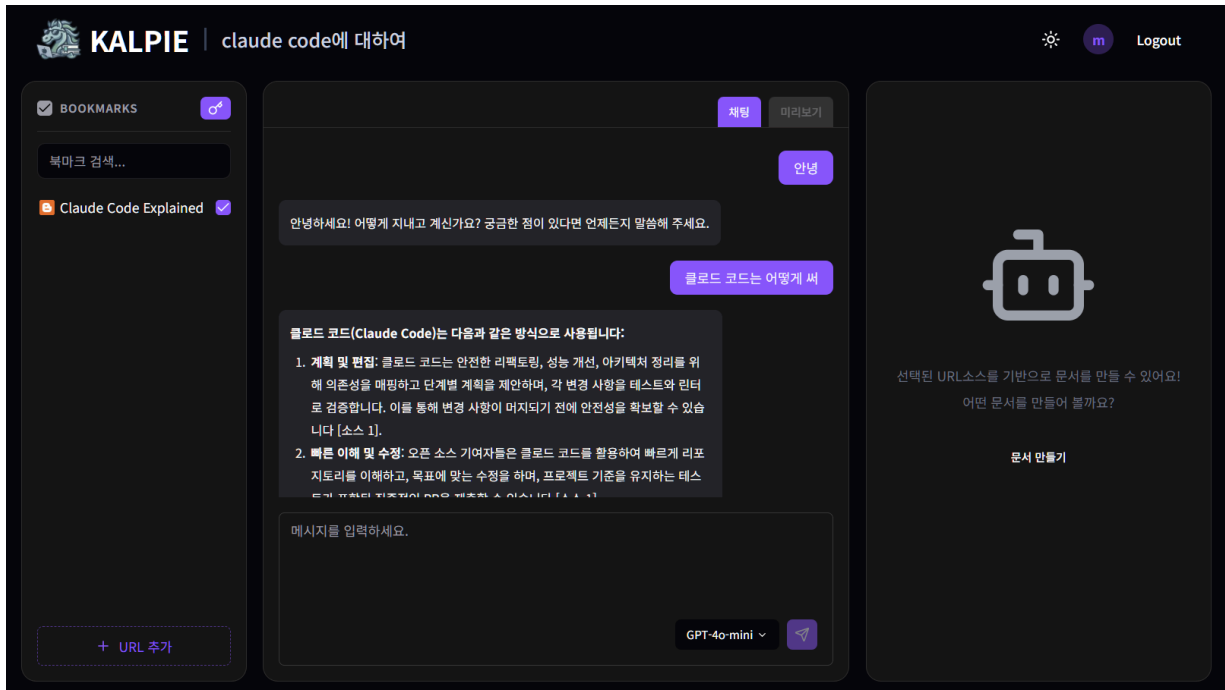


기능

- 새 노트북 생성: "+" 카드 클릭
- 노트북 고정: 핀 아이콘으로 상단 고정
- 정렬: 최근 접근순 / 생성일순 / 이름순
- 컨텍스트 메뉴: 이름 변경, 삭제
- 다크/라이트 모드: 헤더의 테마 토글 버튼

3.3 노트북 화면 (3패널 레이아웃)

노트북을 클릭하면 3분할 패널 화면으로 진입한다.



좌측 패널 — 소스 관리

- 소스 등록: "+ URL 추가" 버튼으로 URL 입력 → 자동 크롤링·임베딩 (진행 상황 실시간 표시)
- 폴더 구조: Chrome 확장 프로그램에서 동기화한 북마크 폴더 구조 유지
- 소스 활성화/비활성화: 체크박스로 RAG 검색 대상 소스 선택
- 전체 선택: 상단 체크박스로 모든 소스 일괄 토글
- 검색: 제목/URL로 소스 필터링
- 편집: 폴더명 변경, 소스 삭제

중앙 패널 — RAG 채팅

- 질문 입력: 하단 텍스트 입력창에 질문 작성 (Enter 전송, Shift+Enter 줄바꿈)
- 모델 선택: 입력창 하단 드롭다운에서 LLM 모델 선택
- 스트리밍 응답: SSE로 실시간 답변 생성, 중간에 Stop 버튼으로 중단 가능
- 출처 인용: 답변 내 [1], [2] 배지 클릭 시 팝오버로 원문 출처(제목, 내용 일부, URL) 확인
- 마크다운 렌더링: 코드 블록, 표, 목록 등 서식 지원

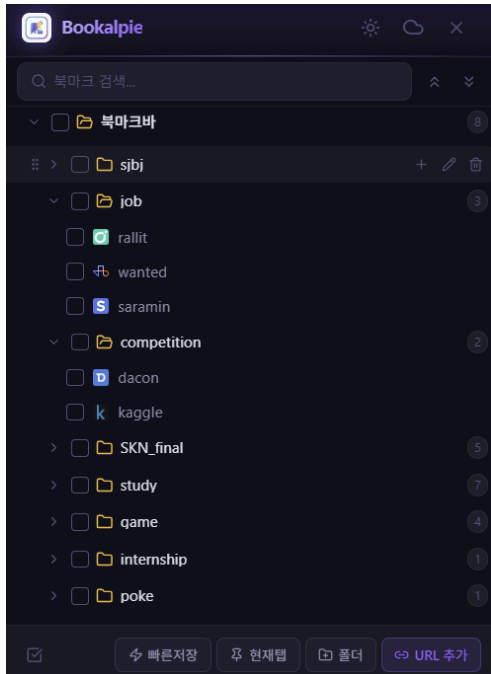
우측 패널 — 리포트 에이전트

- 문서 만들기: 버튼 클릭 시 에이전트 모드 활성화
- 4단계 워크플로우: 1. 요구사항 분석 2. 목차 및 문서 구성 작성 3. 초안 작성 4. 최종 문서 작성
- 단계별 진행 표시: 각 단계의 상태(대기/진행중/완료)와 생성 내용 실시간 표시

- 채팅 모드 복귀: "채팅모드로 돌아가기" 버튼

3.4 Chrome 확장 프로그램 (Bookalpie)

Chrome 북마크를 Kalpie 노트북으로 전송하는 확장 프로그램이다.



빌드

- cd extension
- pnpm install
- pnpm build

Chrome에 로드

- chrome://extensions → 개발자 모드 → 압축해제된 확장 프로그램 로드 → dist/ 선택

사용 흐름

1. Kalpie 웹에서 노트북의 Sync Key 발급
2. 확장 프로그램에서 Sync Key 입력하여 인증
3. 북마크 폴더/항목을 체크박스로 선택
4. "전송" 버튼으로 노트북에 동기화
5. 전송된 URL은 자동으로 크롤링·임베딩 처리

주요 기능

- VS Code 스타일 폴더 트리 탐색
- 드래그 앤 드롭으로 북마크 정리
- 실시간 검색 필터
- 현재 탭 빠른 저장

3.5 API 엔드포인트 요약

기능	메서드	엔드포인트	인증	설명
헬스체크	GET ▾	/api/health	— ▾	서비스 상태 확인
Google 로그인	POST ▾	/api/auth/google	— ▾	OAuth 인증
토큰 갱신	GET ▾	/api/auth/refresh	— ▾	액세스 토큰 재발급
로그인	POST ▾	/api/login	— ▾	이메일/비밀번호 인증
회원가입	POST ▾	/api/signup	— ▾	계정 생성
노트북 CRUD	POST/GET... ▾	/api/notebook/*	✓ ▾	노트북 관리
소스 활성화	PATCH ▾	/api/notebook/{id}/sources/active	✓ ▾	소스 일괄 활성화/비활성
RAG 채팅	POST ▾	/api/chat	✓ ▾	SSE 스트리밍 답변
채팅 이력	GET ▾	/api/chat/notebook/{id}	✓ ▾	노트북별 채팅 목록
리포트 생성	POST ▾	/api/report-workflow/stream	✓ ▾	SSE 스트리밍 리포트
리포트 상태	GET/DELETE ▾	/api/report-workflow/{id}	✓ ▾	워크플로우 조회/초기화
폴더 관리	GET/PATC... ▾	/api/directory/*	✓ ▾	폴더 트리 관리

Sync Key	POST ▾	/api/directory/key	✓ ▾	확장 프로그램 연동 키
북마크 동기화	POST ▾	/api/directory/sync	— ▾	확장 프로그램 → 노트북
소스 추가	POST ▾	/api/source/add	✓ ▾	URL 소스 등록
크롤링 이벤트	GET ▾	/api/source/stream	✓ ▾	SSE 크롤링 진행 상황
크롤링 실행	POST ▾	/crawl (Data Service)	✓ ▾	크롤링 파이프라인 실행

4. 확장 및 커스터마이징

4.1 LLM 모델 교체

Backend의 환경변수로 LLM 엔드포인트를 교체할 수 있다.

환경변수	대상	현재 설정
OPENAI_API_KEY	주 추론 모델 (GPT-4o-mini)	OpenAI API ▾
CUSTOM_LLM_MODEL_URL	대체 모델 (EXAONE-4.0-32B)	RunPod vLLM ▾
EMBEDDING_MODEL_URL	임베딩 모델 (bge-m3)	RunPod Serverless ▾
RERANKER_MODEL_URL	리랭커 (bge-reranker-v2-m3)	RunPod Serverless ▾

LangGraph 에이전트의 모델 선택은 `backend/agent/model/` 디렉토리에서 관리하며, 프론트엔드의 모델 선택 드롭다운과 연동된다.

4.2 RAG 파이프라인 커스터마이징

LangGraph 기반 에이전트 노드를 개별적으로 수정할 수 있다.

```
backend/agent/
├── nodes/
│   ├── classify_question.py # 질문 3분류 (simple/complex/comparison)
│   └── decompose_query.py # 복합 질문 분해
```

```

|   ├── rewrite_query.py          # 검색 쿼리 재작성
|   ├── retrieve_sources.py       # 벡터 검색 (k=10) + 리랭킹
|   └── generate_answer.py        # LLM 답변 생성 + 출처 인용
└── model/
    ├── llm.py                   # LLM 인스턴스 설정
    ├── embedding.py             # 임베딩 모델 설정
    └── reranker.py               # 리랭커 설정
└── graph.py                     # LangGraph 워크플로우 정의
└── state.py                     # 상태 스키마

```

커스터마이징 예시:

- 검색 결과 수 변경: `retrieve_sources.py`의 `k` 파라미터
- 분류 기준 변경: `classify_question.py`의 프롬프트
- 답변 스타일 변경: `generate_answer.py`의 시스템 프롬프트

4.3 데이터 파이프라인 커스터마이징

Data Service의 크롤링·청킹·임베딩 파이프라인을 조정할 수 있다.

```

data/
├── services/
│   ├── crawl/                  # 크롤링 (Trafilatura + Playwright)
│   ├── chunk/                  # 청킹 (HierarchicalPrepend 방식)
│   ├── llm/                    # LLM 정제/요약
│   └── embed/                  # 임베딩 생성
└── core/
    └── config.py               # 워커 수, 모델 URL 등 설정

```

커스터마이징 예시:

- 청킹 사이즈 조정: 청크 분할 파라미터 변경
- 크롤러 변경: `Trafilatura/Playwright` 선택 로직 수정
- 동시 처리 수: `MAX_WORKERS` 환경변수

4.4 프론트엔드 UI 커스터마이징

영역	파일 위치	설명
테마	<code>frontend/src/app/globals.css</code>	CSS 변수 기반 다크/라이트 테마

레이아웃	frontend/src/containers/notebook/	3패널 비율 조정 (react-resizable-panels)
컴포넌트	frontend/src/shared/components/ui/	Radix UI 기반 공통 컴포넌트
상태관리	frontend/src/shared/store/	Zustand 스토어 (노트북, 채팅, 에이전트 등)

4.5 리포트 워크플로우 커스터마이징

backend/services/report.py에서 4단계 리포트 생성 워크플로우를 수정할 수 있다.

단계	역할	커스터마이징 포인트
Step 1	요구사항 분석	분석 프롬프트 수정
Step 2	목차 구성	문서 구조 템플릿 변경
Step 3	초안 작성	작성 스타일/톤 조정
Step 4	최종 문서	검토 기준 변경

각 단계는 LangGraph의 interrupt 기반 사용자 리뷰 루프를 지원하여, 중간 결과를 확인하고 수정 요청을 반영할 수 있다.